Peeky

Let AI agents look inside running programs, and never mislead them

Why it exists · What it is · How it works

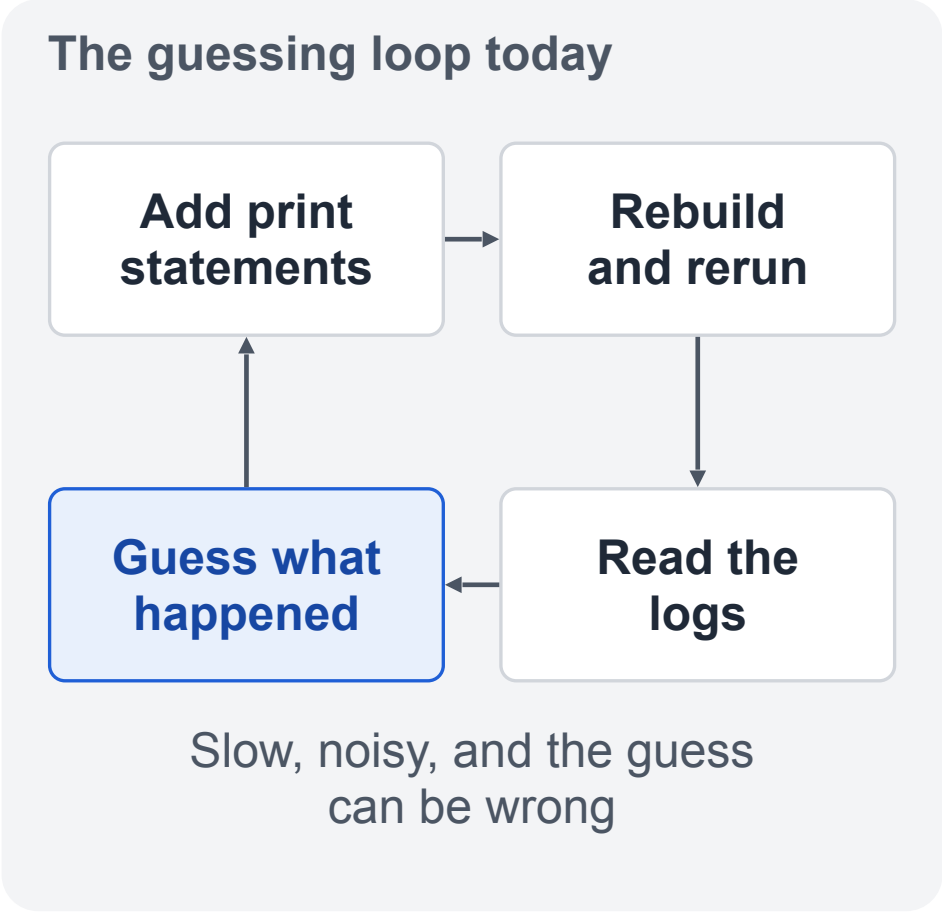
Python **Java** **C** **C++** **Rust**

Briefing for the executive team and engineering teams · October 2026

github.com/sushantmane/peeky · version 0.1.0, released 26 October 2026

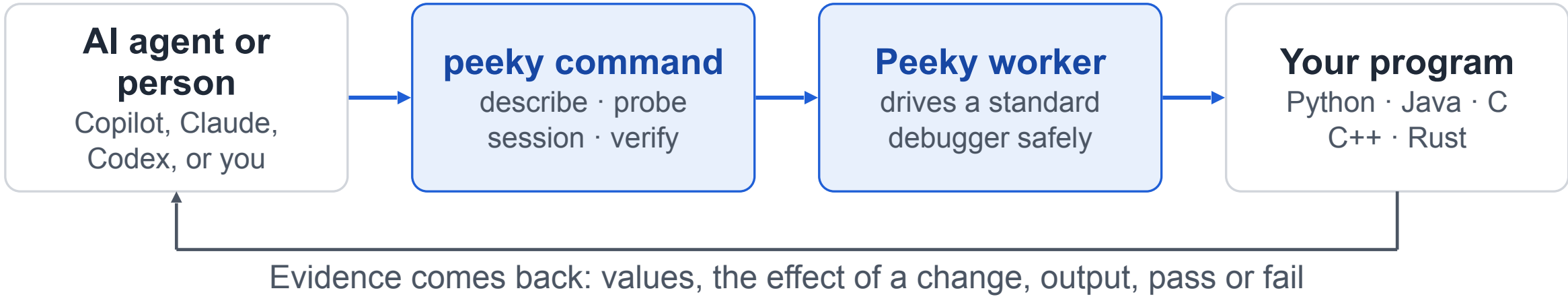
AI coding agents guess at what programs do

- Agents read code and logs, then **infer** what happened at run time
- To see one value, they add print statements, rebuild and rerun
- Debuggers are built for a person at a screen, not for an agent's tools
- A **wrong value** is worse than none: the agent builds its fix on it
- "Fixed" is often claimed without an **independent check**



Peeky in one sentence

A debugger that AI agents and people run from the command line. It **stops** a program, **reads and changes** its values, **tries a fix** on a fresh copy, and **checks** it with your own test.



Peeky **never calls an AI model**: the agent decides, Peeky reports what it saw.

The core promise

Never a wrong or guessed value without a note.

Withhold, and say why

If Peeky can't prove a value, it shows none, explains, and says what to do instead.

```
no value yet:
declared at line 18,
below this stop
```

Mark only the proven

A **[settable]** mark appears only where a change reaches what the program goes on to use.

```
start  3  [settable]
```

Verified means checked

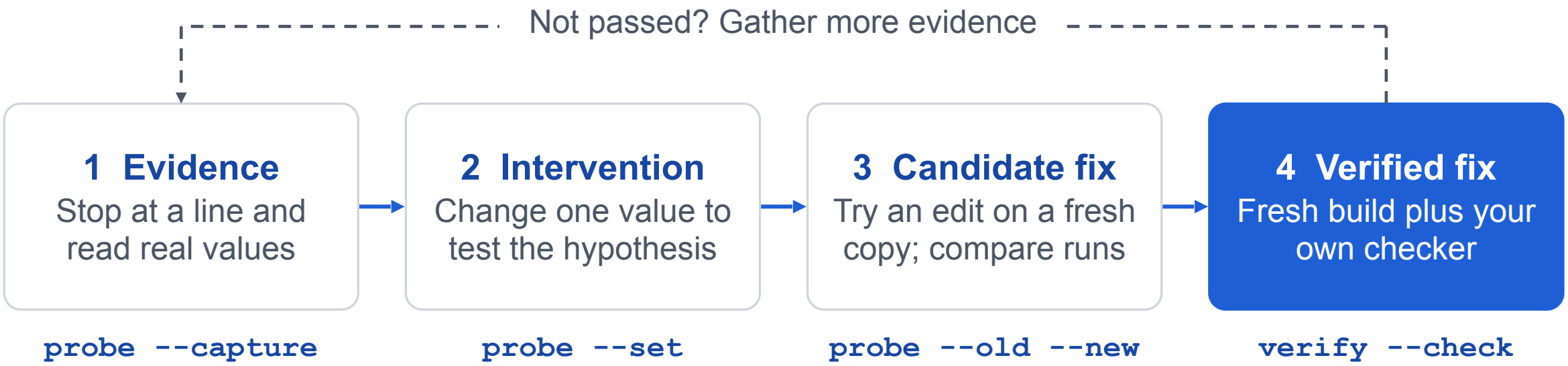
A fix passes only on a fresh build, with your own independent checker.

```
passed: True
stage: verification
```

Refuse rather than mislead. Every refusal says what to do instead.

THE AGENT'S LOOP

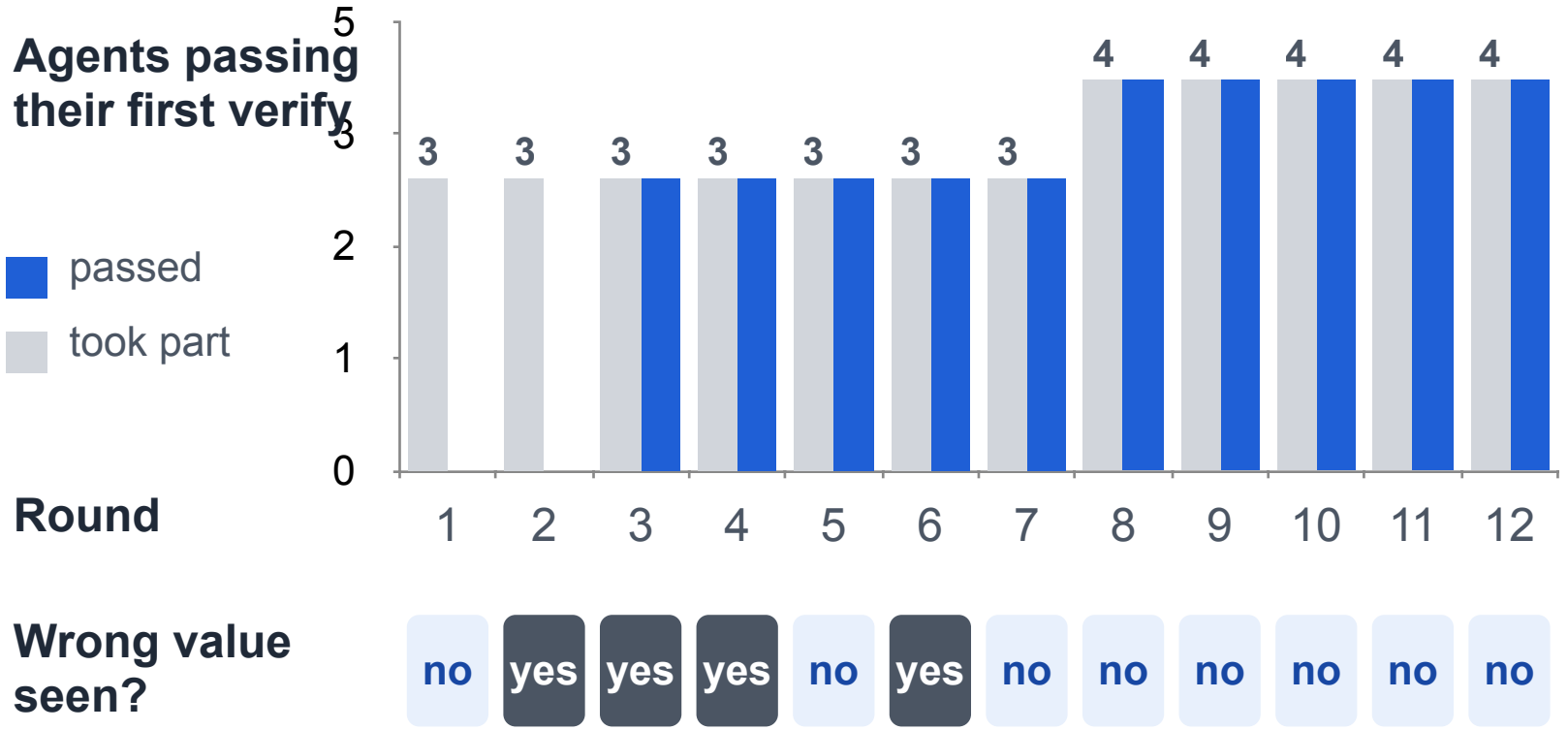
How an agent uses Peeky



Steps 1 to 3 are **experiments**: they produce evidence.
Only step 4 says whether a fix **passes**. Each step is one command.

EVIDENCE THAT IT WORKS

Results: twelve rounds of live agent evaluations



n/r: not recorded. Round 2: first fixes special-cased the input and were rejected; all three then verified general fixes. Wrong values: rounds 1–6 from findings tables.

35 of 35

agents passed their first verify, rounds 3 to 12

0

wrong values reached an agent in rounds 7 to 12

5

languages covered: Python, Java, C, C++ and Rust

WHERE WE ARE

Status and roadmap

Delivered

- Version 0.1.0, released 26 Oct 2026
- All five languages
- Remote worker over SSH
- Evidence store with SQL

Since 0.1.0

- GCC C++: unordered containers, lambda captures
- Static locals, caller aggregates
- More string sets and reference bindings

Next

- Steps to a public release
- Thirteenth agent evaluation
- CUDA phase 0 on a real GPU host

Deferred

- Kubernetes workers and relay
- Several users per worker
- Attach beyond Java
- GDB backend (~4–6 weeks)

Since 14 Oct 2026: **234** merged pull requests · **~4,100** unit tests · **14** Linux test lanes on every merge

Next steps toward a public release

STEP 1

Choose a licence

0.1.0 ships with no licence. The bundled Java debug adapter is EPL-1.0.

→ **Pick one that fits shipping that adapter.**

STEP 2

Publish to PyPI

"peeky" on PyPI is an unrelated project. Today: a wheel on the release.

→ **Publish under a package name of its own.**

STEP 3

CI on x86-64

Every merge is checked by 14 Linux lanes in Docker on Apple Silicon.

→ **Run the same lanes on x86-64 once public.**

STEP 4

The thirteenth evaluation

Fresh agents, new bugs, docs only: 0.1.0 plus the new GCC C++ reads.

→ **Fix its findings before the public release.**

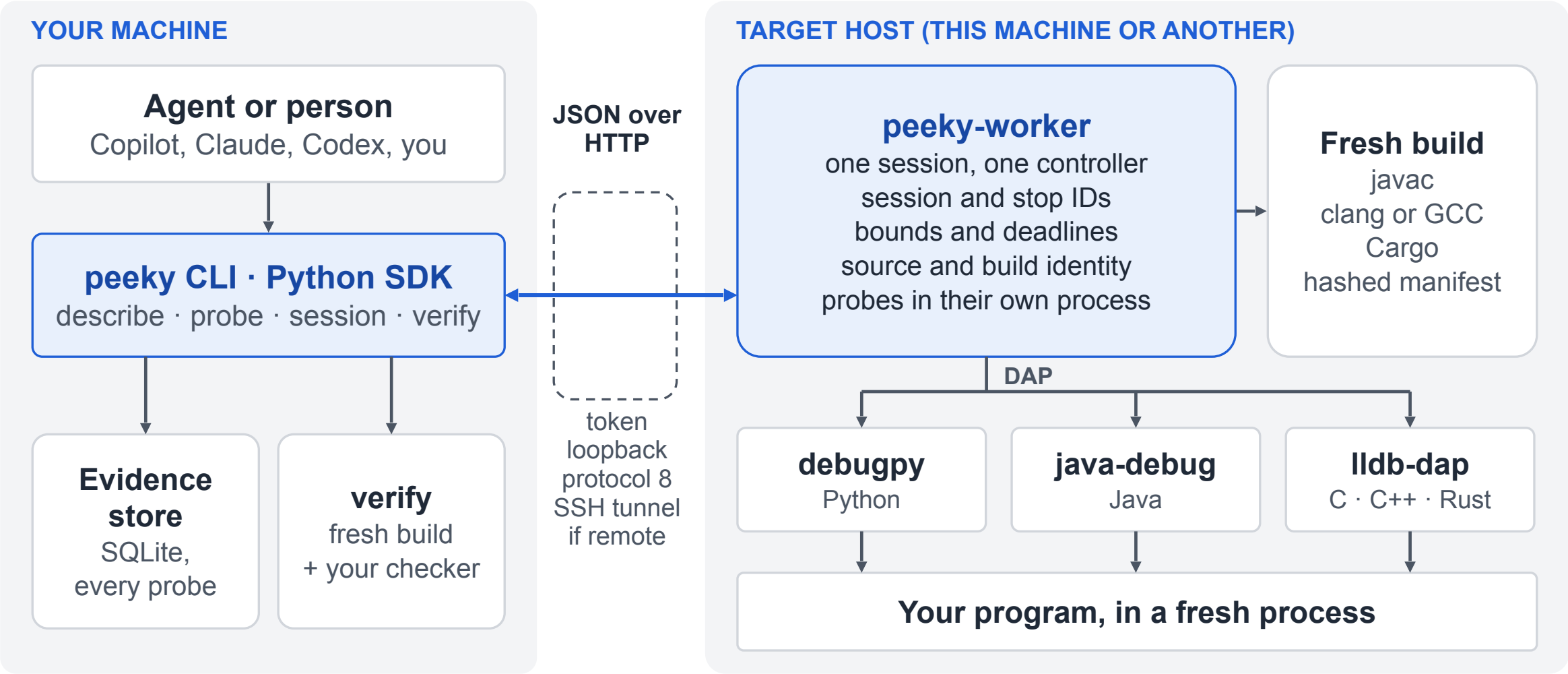
PART 2

Engineering: how Peeky works

1. Architecture and the two ways to ask
2. Design decisions and the truth model
3. Evidence store, candidate edits and verify
4. Remote workers and the threat model
5. Languages, quality process and tests
6. Performance, limits and getting started

HOW IT FITS TOGETHER

Architecture overview



ADR 0004

Two ways to ask: probes and sessions

Probe: one request, one answer

Build → Launch → Stop → Read

Change → Run to end → Clean up

- No IDs cross the boundary: a retry is a new run
- Pauses of milliseconds per hit: safe for timed code
- Saved to the evidence store for follow-ups

Session: several looks at one stop

start → inspect → get → set

step → continue → close

- Stop IDs and handles revoked on every move
- A lost reply closes the session: no blind retry
- A pause limit auto-resumes a stale stop

One guarded engine: a probe is the worker's own operations, composed inside the worker.

WHY IT IS BUILT THIS WAY

Key design decisions

Tool, not agent

Never calls a model. Agents call it like git; people run the same commands.

ADR 0005

Single ownership

One controller, one session per worker: IDs and cleanup stay simple.

ADR 0001

Bounded everything

4 MiB requests; 64 values a page, depth 4, 512 nodes; deadlines on every step.

DESIGN

No blind retries

A lost reply to a change closes the session. A probe retry is a fresh run.

DESIGN

Provenance checks

Sources, builds, toolchains and adapters are hashed, then rechecked.

THREAT MODEL

No code in the program

Selectors are paths, not expressions. Conditional breakpoints refused.

DESIGN

The truth model: withhold, mark, refuse

```
start    3           [settable]
cents    4500        [not settable: register]
label    <no value yet: not assigned
          on every path to here ...>
ordered  Vec<int, …> @ 0x16fc46ef8 [note]
```

Rows abridged from the Rust and C/C++ cookbooks

In Rust, the **[settable]** promise covers the paths that don't panic or loop forever.

- **Withhold** what can't be proven: say why, and what to do instead
- **[settable]** promises the change reaches what the program goes on to use
- **Refuse before the run** when the build rules a change out
- **Read back** every change; a mismatch closes the session
- **Opaque stays opaque:** no layout decoded by guess
- **Hedge** a note where the proof is partial

Rust: proving a change will be seen

Unoptimized Cargo build
plus a MIR dump and LLVM IR

↓

Map the stop line to MIR
through the IR's debug locations

↓

Dataflow on every path
moves, drops, copies, borrows, calls

↓

A verdict per value
read, [settable], withheld or refused

A real refusal, before the program runs:

```
$ peeky probe examples/rust --at 36 \
      --set request.page=2
...the program has already copied `request`
into `start` (at line 35) and goes on with
that copy ... settable here: start
```

- Owned `Vec`/`String` contents are read only if initialized on every path
- A change is refused if a copy is under way, or no path reads it again
- **Anything the analysis doesn't understand proves nothing**

C and C++: proofs from debug info and machine code

- Place each frame's function from DWARF; **if unsure, withhold the frame**
- Withhold locals not yet declared: below the stop, or on a line not started
- Read machine code and exception tables to tell if a line has started
- Change integers only, at a statement's start, in the owning frame
- Pointers: address facts only; **one past the end is withheld**
- GCC builds: unordered containers and lambda captures read; the unproven withheld

Evaluation round 6, before the fix

```
qty   6712214884   [settable]
```

Leftover memory shown as a value: the core rule broken



Today the rule withholds it

```
qty   <no value yet: declared
       at line 23, below this stop>
```

Row format from the C and C++ cookbook

Java and Python, briefly

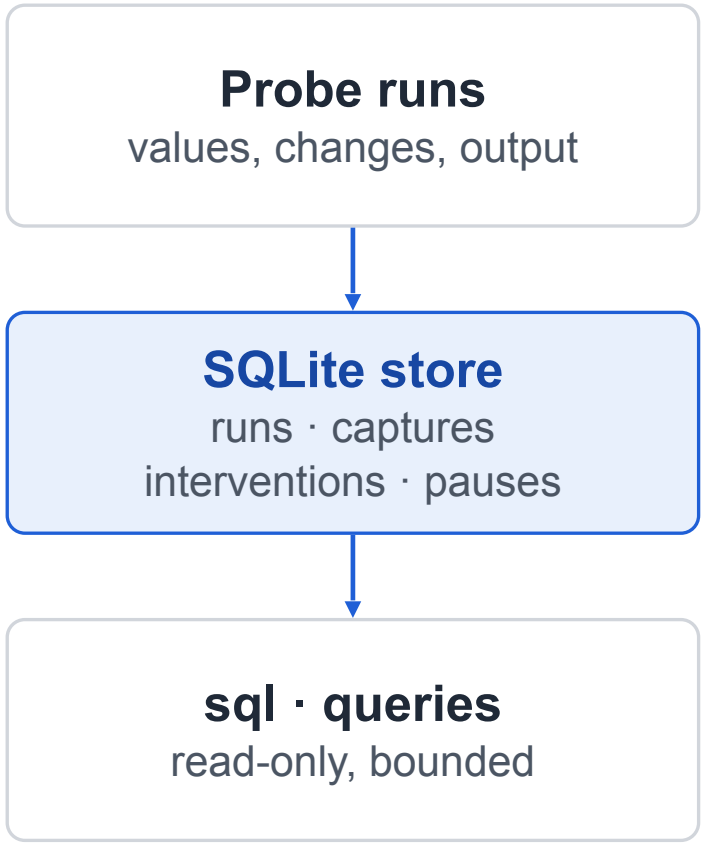
Java `java-debug over JDI/JDWP`

- Fresh javac build; source maps from class files
- Changes locals, fields, array elements; values must fit the type
- Never changes the JDK's own state
- `[maybe settable]` when other threads could interfere
- Attaches to a Gradle `--debug-jvm` test

Python `debugpy`

- `--set` takes literals only; never runs an expression
- `[id 19]`: the same number means the same object at one stop
- Reads a global only where the function provably uses it
- A loop leftover gets a note
- Past debugpy's 100-item window: "beyond", not "missing"

The evidence store: capture once, ask many questions



- Saved locally by the client; newest 500 runs kept
- Read-only SQL: 10 s, 16 MB, at most 10,000 rows
- Ready-made queries: `diff`, `compare`, `join`, `set_effect`…

`queries diff` leads with a verdict:

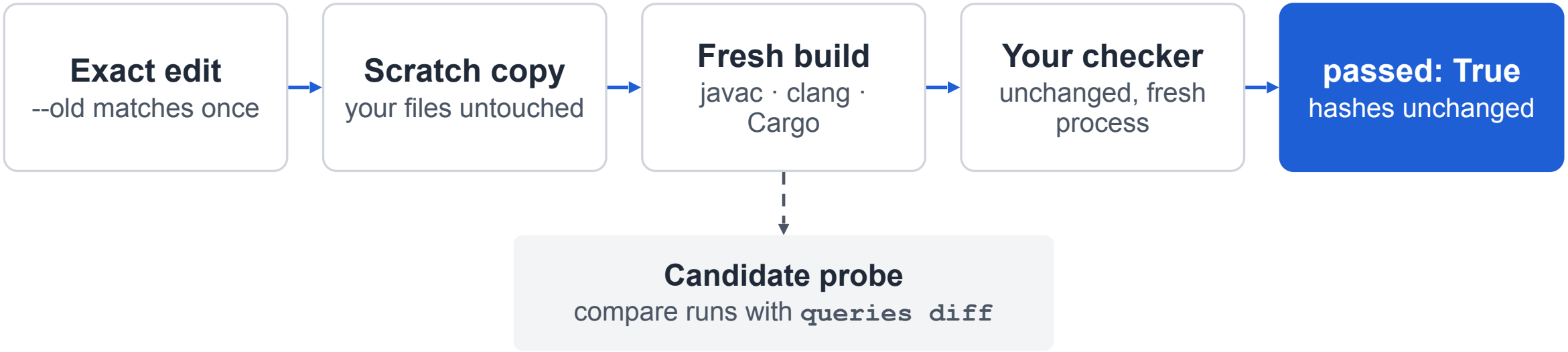
hits pair by `i`

hits pair as far as `k` shows

hits may not line up: …

No verdict ever claims two runs' hits are the **same moments**.

Candidate edits and verify



- Run `verify --baseline` first: the checker must **fail** on the unedited program
- The checker must finish: an early `exit(0)` is rejected
- Changed original or checker hashes: **no pass.** Rust also runs its hashed Cargo test
- A pass is only as broad as the checker

Remote workers and the threat model



- Every source compared before launch; a mismatch is refused
- Different protocol versions refuse each other
- Token stripped from debugger and checker processes

- **Trusted code only:** not a sandbox, not a production service
- Hashes catch accidents, not an attacker running as you
- CI drives a second host through a real `ssh -L`

Verification runs on the client: its toolchain must match the target's.

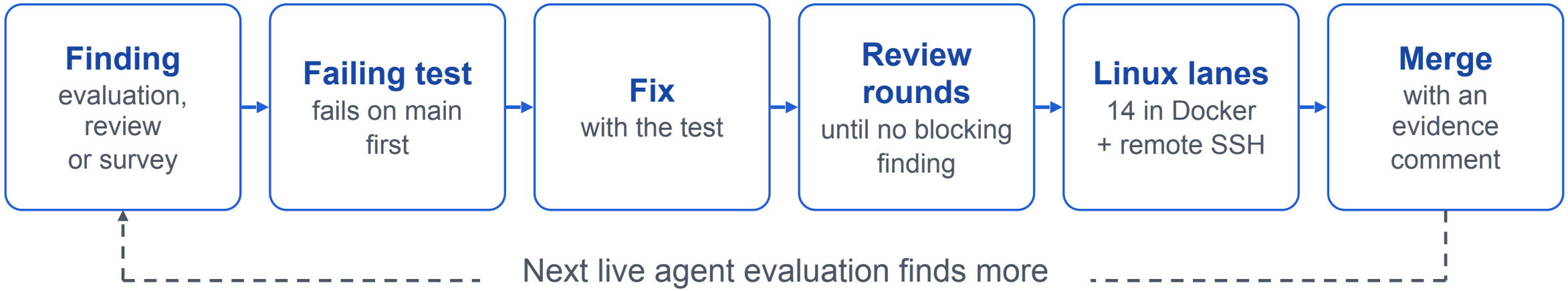
Per-language implementation

Language	Debugger	Build and identity	Reads and changes
Python	debugpy 1.8.22	No build; sources hashed	Literals only; object `[id N]`
Java	java-debug, headless	Fresh javac; maps from class files	Fields, array elements, Strings
C	lldb-dap 18 / 20	clang or GCC; headers rechecked	Integers in the owning frame
C++	lldb-dap 18 / 20	C++17, clang or GCC	Classes, std containers, strings
Rust	lldb-dap 18 / 20	Cargo 1.96; MIR and IR too	Proven Vec/String; proven changes

Every language: its own review, real-adapter conformance tests and CI lane before support is claimed. All five qualified on a separate host.

EVERY FIX, EVERY TIME

Quality process: how every fix lands



Review until nothing blocks

The latest Rust rule took five rounds; each blocking finding got a test that failed on the reviewed head

One wrong value, fixed first

Round 6 found a C++ leftover shown as a value; fixed in its own pull request before round 7

Test matrix and CI lanes

Lane	What it runs	Tests
Unit	Framing, failures, lifecycle, analyses, queries	~4,100
Integration	Real debugpy over HTTP	218
C	clang 18; GCC with lldb-dap 20	~540
C++ × 4	clang or GCC, with lldb-dap 18 or 20	307–444
Rust × 2	Rust 1.96, with lldb-dap 18 or 20	~300
Java	JDK 17, real JVM and adapter	~140
Remote SSH	A second host through ssh -L, five languages	21
Also	Lint, wheel smoke test, GDB DAP sequence	—

14

Linux lanes in Docker on every merge

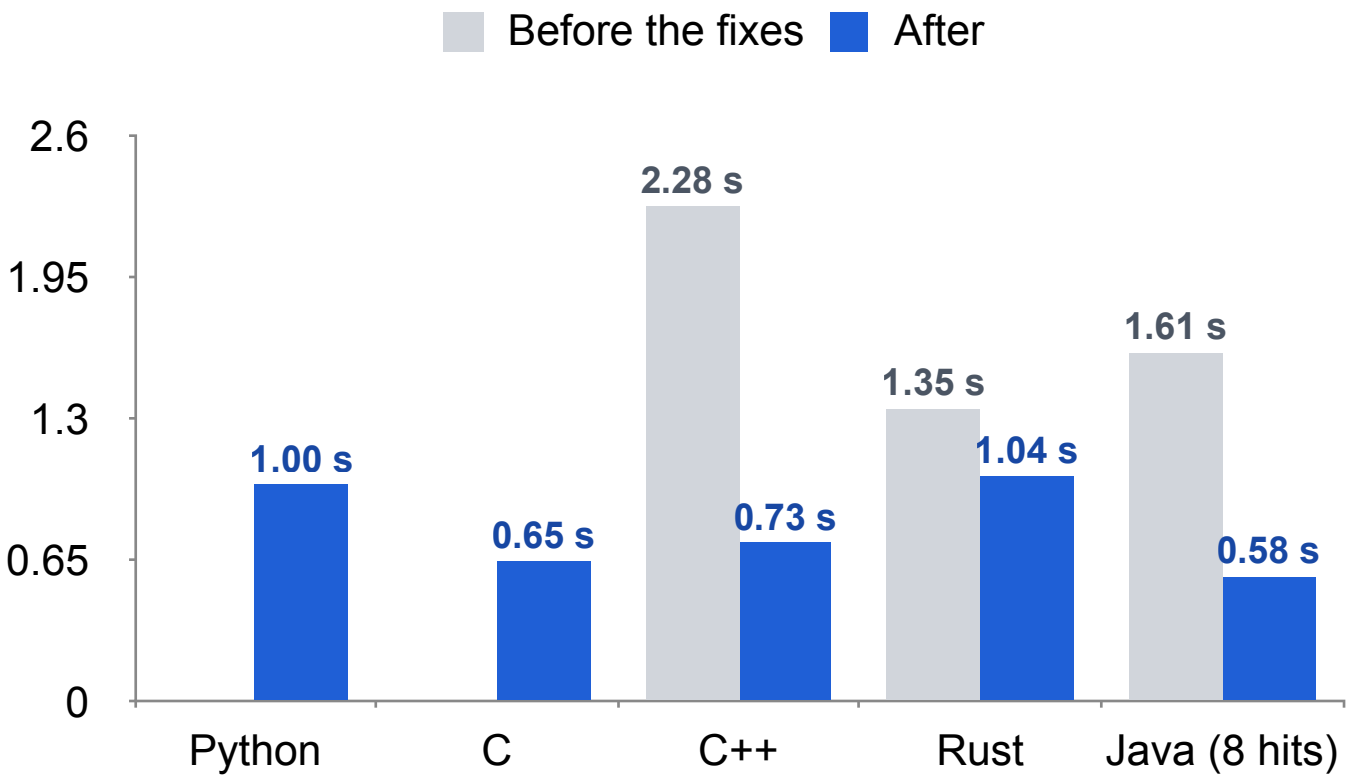
0

live models, tokens or paid services needed in CI

Counted at commit bbed6e4; numbers rounded

Performance: where the time goes

One probe's wall time, seconds



- Most time is compilers, adapters and the program
- Peeky's own CPU: ~0.5% of a Python probe, 10–15% of a compiled one
- The fix: stop rehashing unchanged toolchains
- Java pause per hit: 120 ms → 9.9 ms
- So no rewrite in Rust: **measure first**

One Apple Silicon Mac, small samples: scale, not service-level promises. C's probe failed before its fix; Python has no before figure.

HONEST BOUNDARIES

Known limits and deferred work

Limits today

- Experimental in all five languages
- GCC C++: some unordered entries withheld
- Rust: one dependency-free Cargo binary
- verify edits one file; attach is Java-only
- CI: Linux lanes on arm64; no macOS yet

Deferred on purpose

- CUDA and mixed Python/C++/GPU stacks
- Kubernetes workers; several users per worker
- Production mutation and data policies
- GDB backend; build cache only if measured
- Reverse debugging and replay

Round 12's three Medium issues are fixed. **The thirteenth evaluation is next.**

TRY IT

Getting started

```
# Python 3.11+, from the repository root
pip install -e .

# Ask one question: stop, read, change
peeky probe examples/pagination_bug.py \
  --at 8 --capture 'start,items[1]' \
  --set start=0

# Check a fix with the program's checker
peeky verify examples/pagination_bug.py \
  --check examples/verify_pagination.py \
  --old 'page * page_size' \
  --new '(page - 1) * page_size'
```

Expect `set start: 3 -> 0`, then `PAGE: [10, 20, 30]`; verify prints `passed: True`.

Agents

`docs/AGENT_GUIDE.md`
`skills/peeky-debug`

People

`docs/COOKBOOK.md`
`docs/lang/*.md`

Errors and remote

`docs/ERRORS.md`
`docs/REMOTE.md`